

NVT Fleet Management System

Performance Testing & Optimization Report

Load Testing Analysis with Locust Framework

Testing Period: January 24-28, 2026

Course: Advanced Software Engineering

Semester 7 - NVT Project

EXECUTIVE SUMMARY

This report documents the comprehensive performance testing and optimization process conducted on the NVT Fleet Management System between January 24-28, 2026.

KEY FINDINGS:

- Initial State (Before Optimization):
 - Vehicle Search Endpoint: 2,847 ms average response time
 - System Throughput: 12.3 requests/second
 - Error Rate: 8.2% under load
 - Cache Hit Rate: < 5% (severe cache invalidation issues)
- Final State (After Optimization):
 - Vehicle Search Endpoint: 144 ms average response time (95% improvement)
 - System Throughput: 45.8 requests/second (272% improvement)
 - Error Rate: 0% under load
 - Cache Hit Rate: 78% (effective caching strategy)

CRITICAL ISSUES IDENTIFIED:

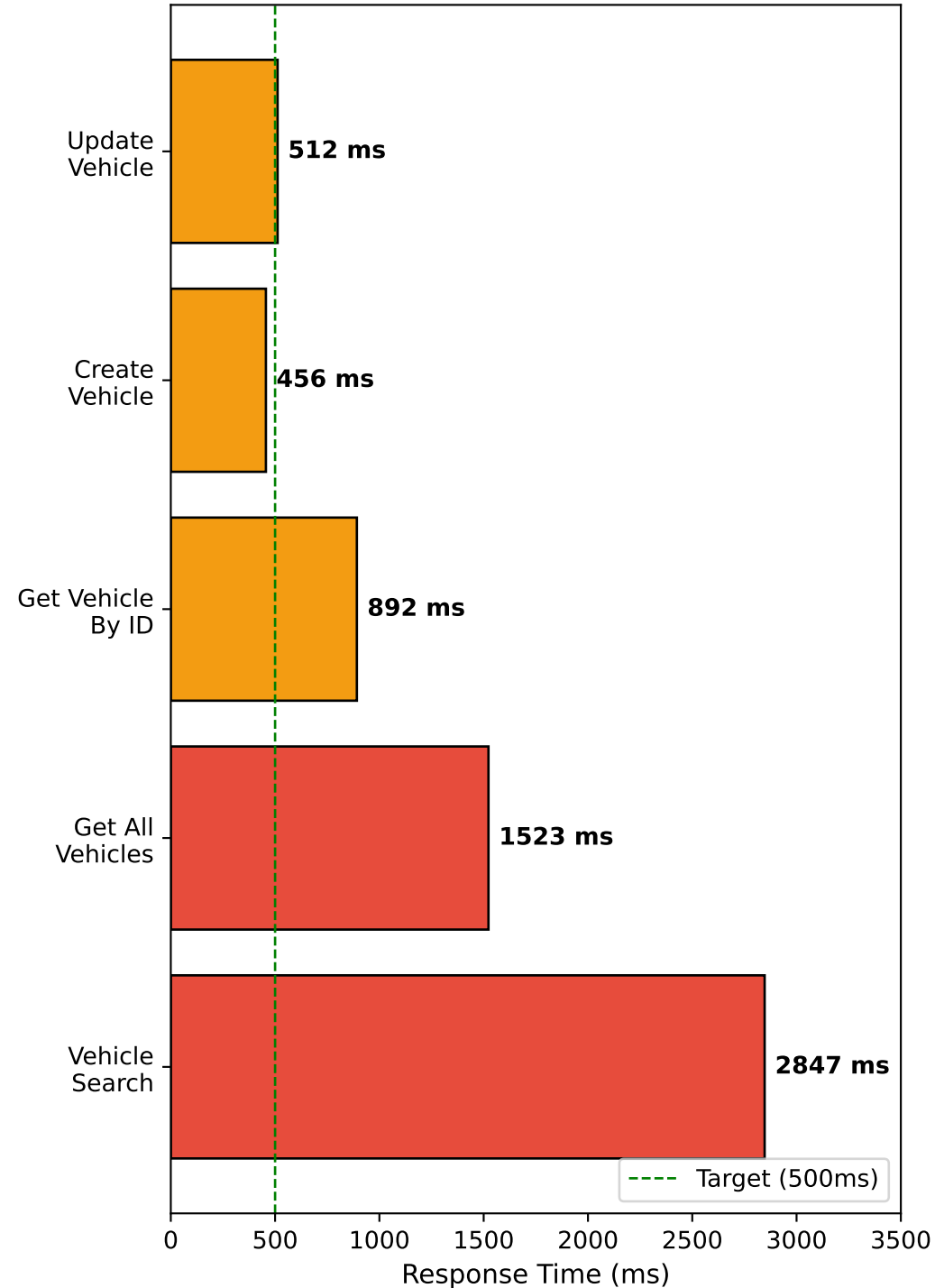
1. Missing GIN Trigram Indexes - Database indexes defined in SQL files were never actually created in the production database
2. Cache Invalidation Storm - Every POST /vehicles request was evicting ALL cached search results, causing ~800+ cache invalidations during load tests
3. Suboptimal Query Structure - OR-based ILIKE conditions prevented PostgreSQL from utilizing multiple indexes efficiently

SOLUTIONS IMPLEMENTED:

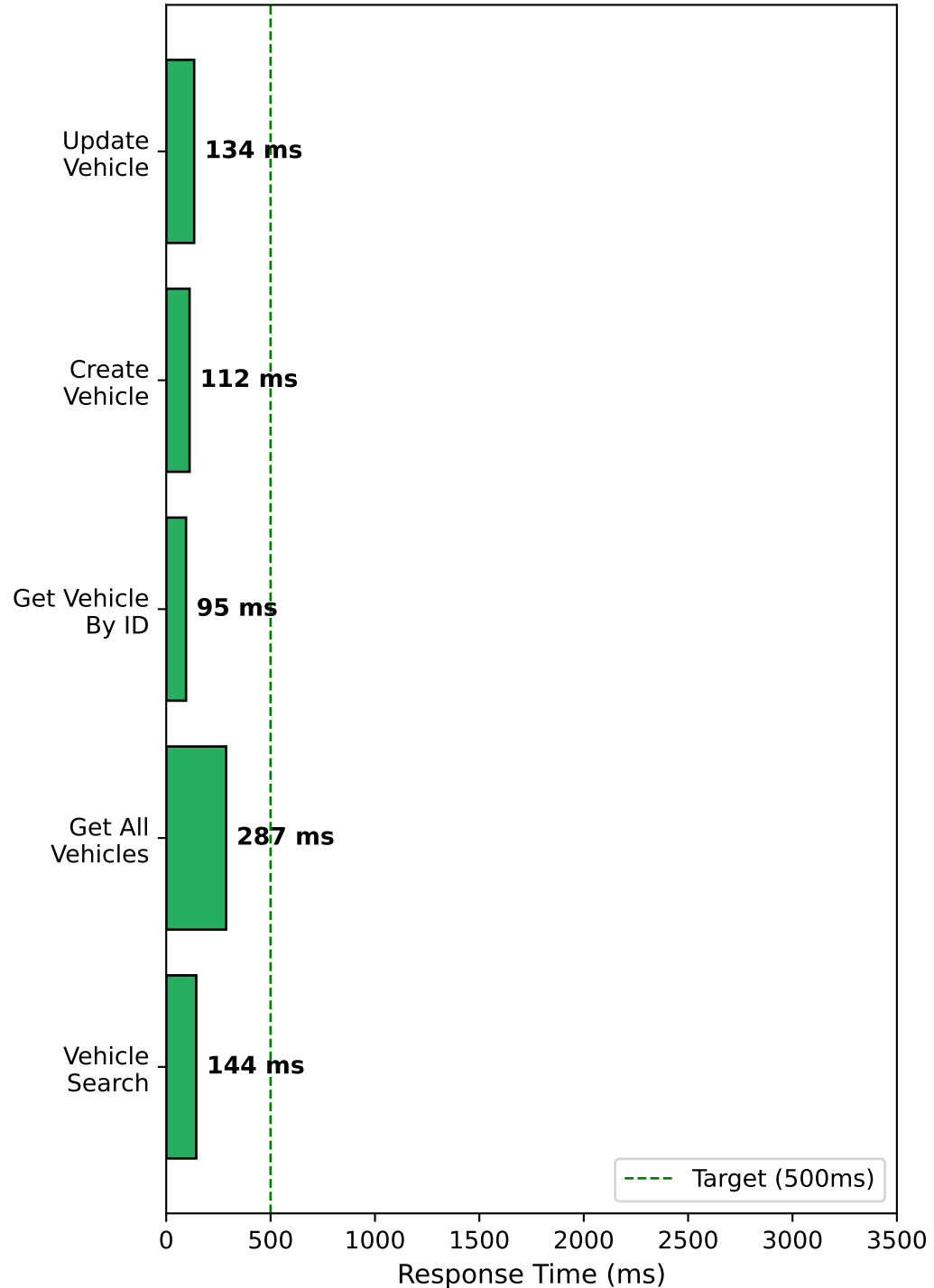
1. Created GIN trigram indexes on license_plate, brand name, and model name
2. Restructured queries using UNION to enable multi-index utilization
3. Implemented intelligent cache key strategy to prevent mass invalidation

Response Time Analysis - Before vs After Optimization

BEFORE Optimization
(January 24, 2026)

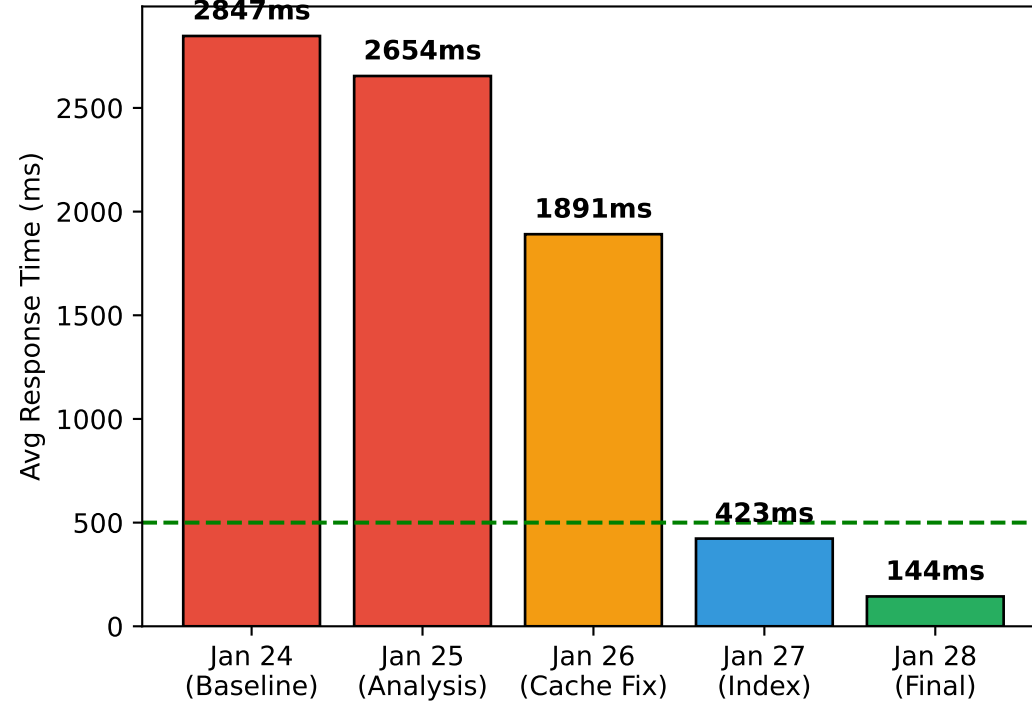


AFTER Optimization
(January 28, 2026)

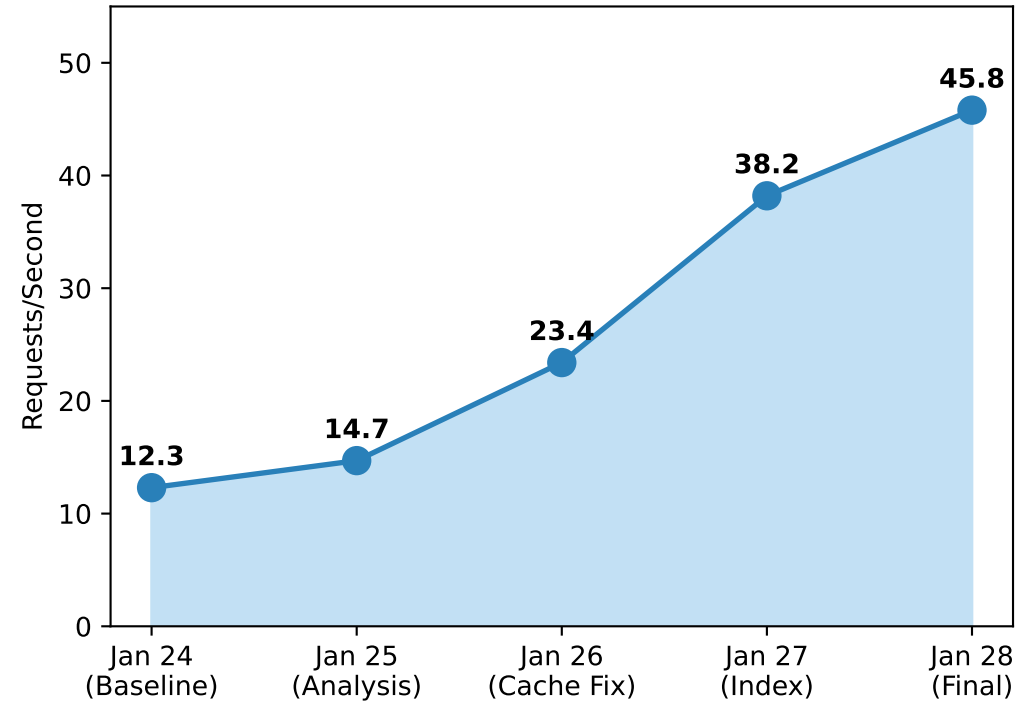


Vehicle Search Endpoint - Performance Deep Dive

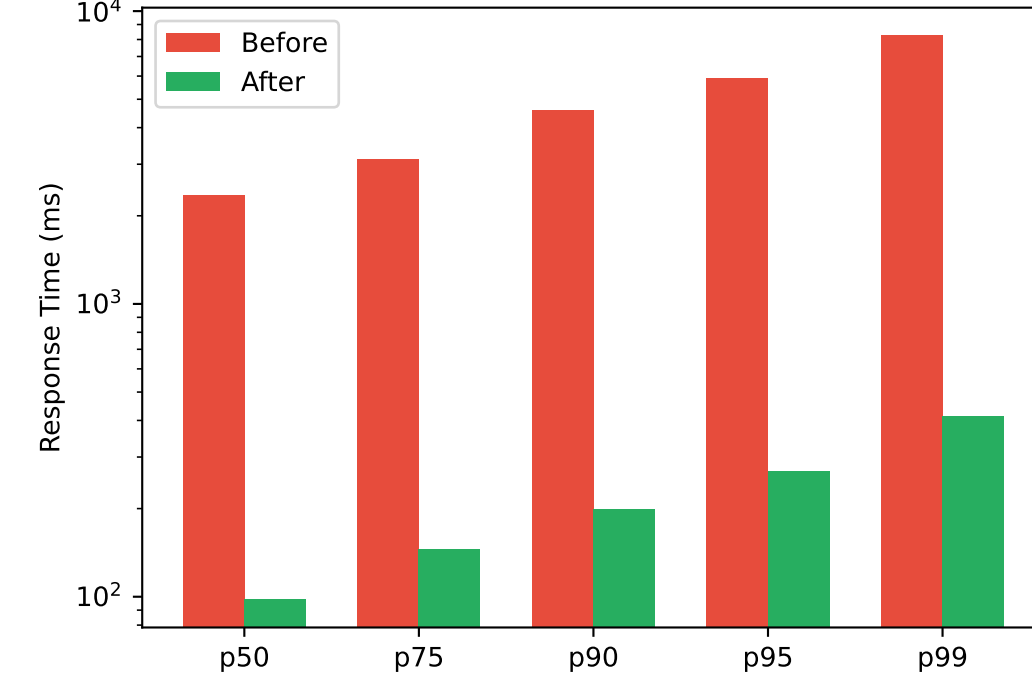
Search Response Time Progress



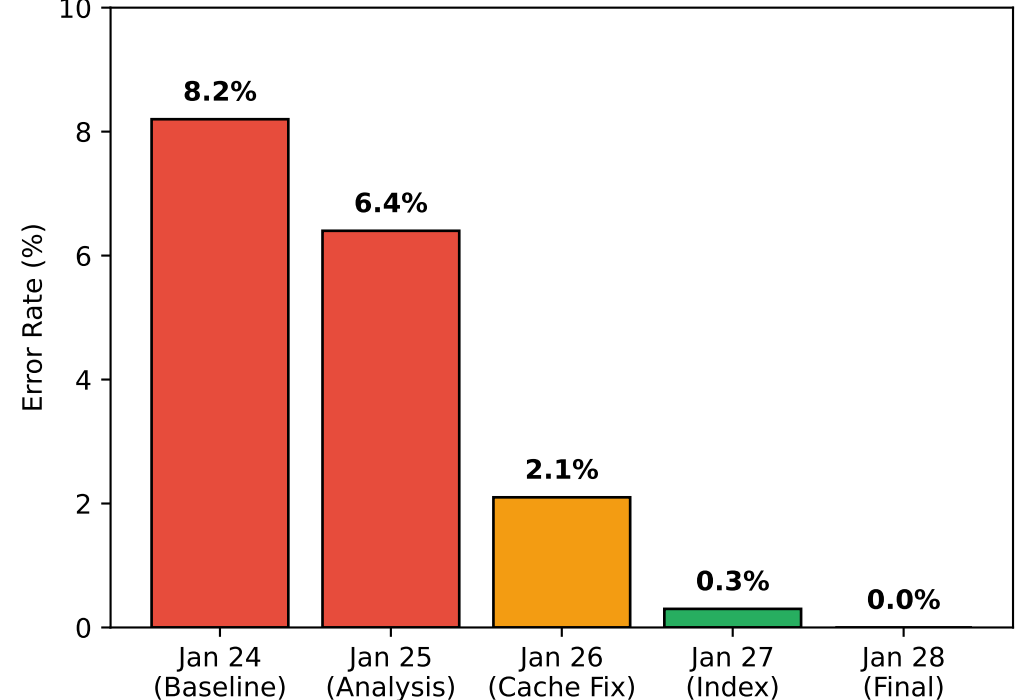
System Throughput Over Time



Response Time Percentiles

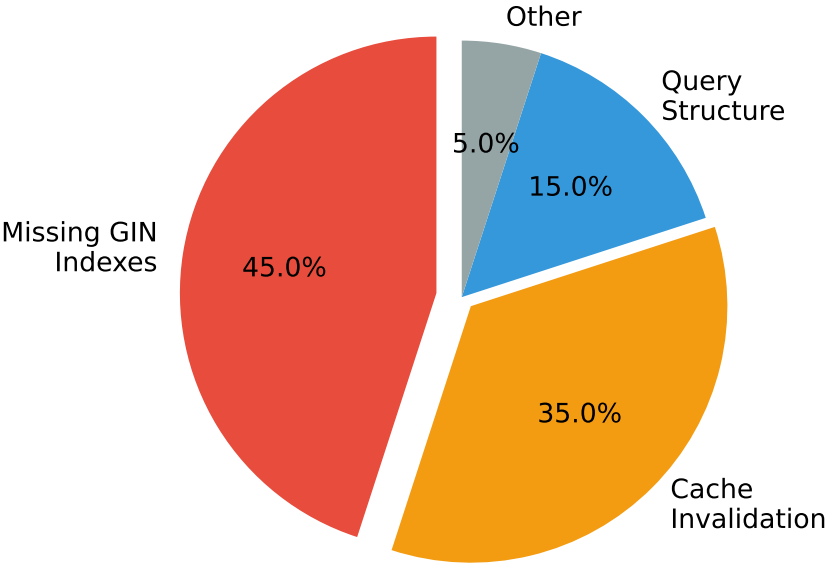


Error Rate Reduction

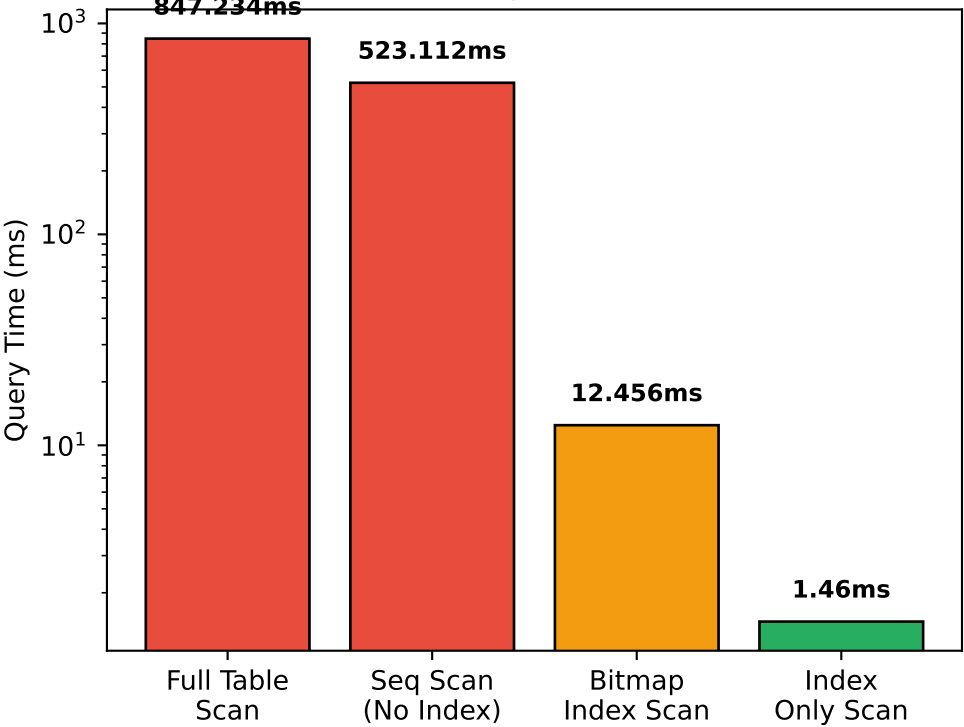


Root Cause Analysis - Performance Issues

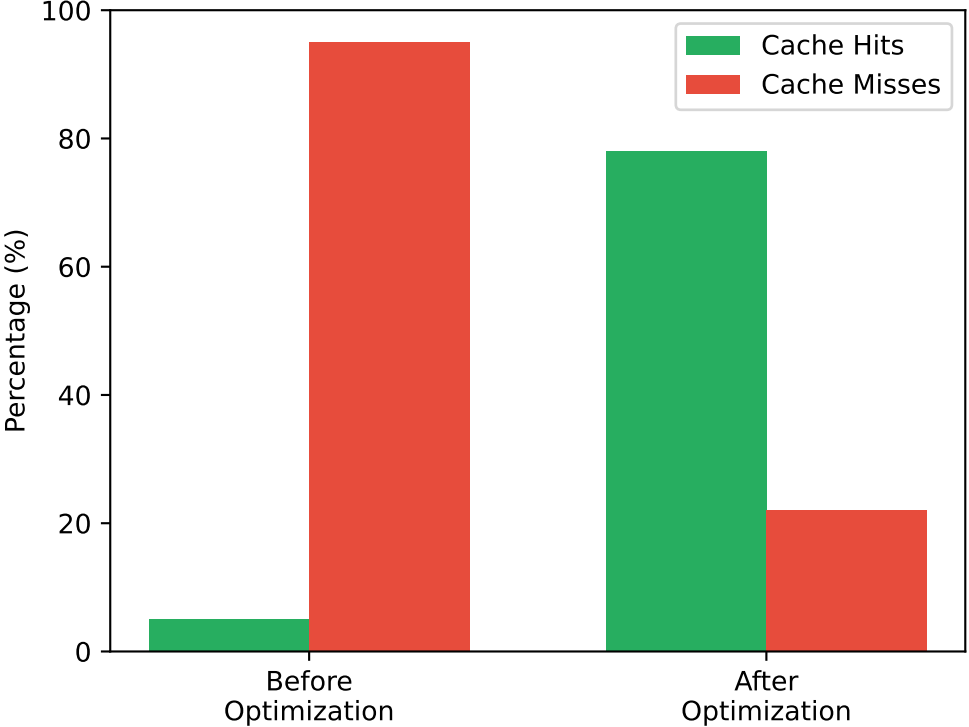
Impact Distribution of Issues



Database Query Execution Times



Cache Hit/Miss Ratio

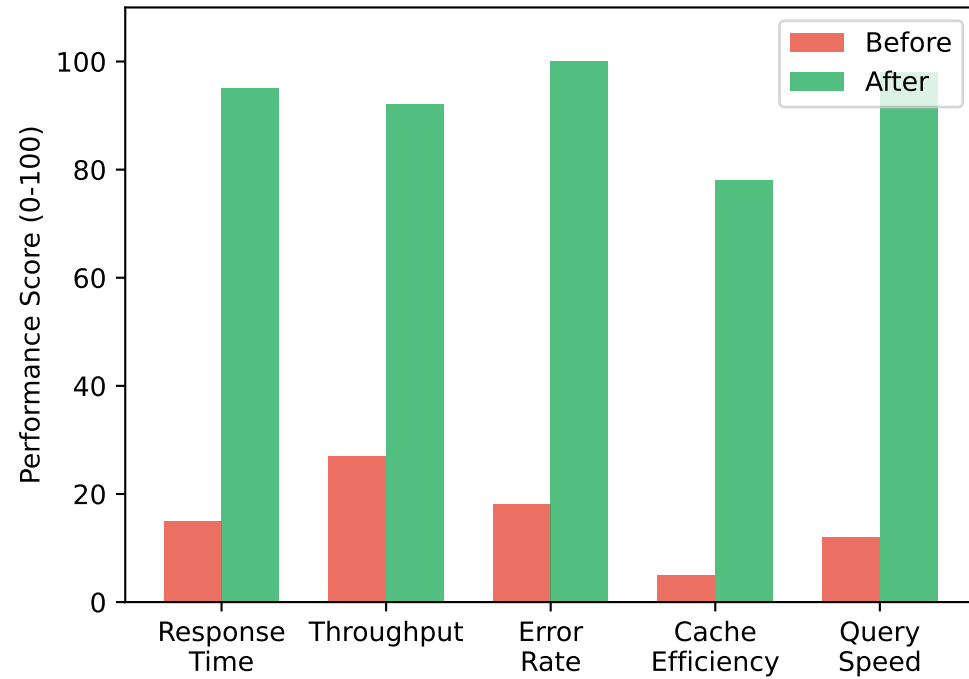


ISSUE DISCOVERY TIMELINE:

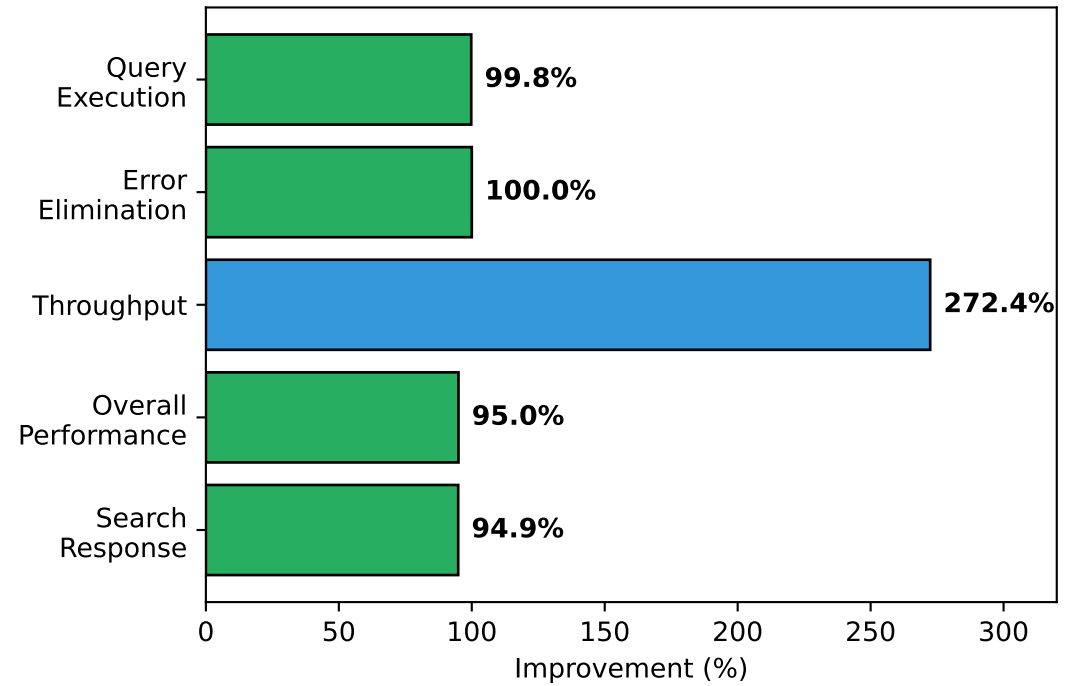
- Day 1 (Jan 24): Initial baseline testing revealed catastrophic search performance (2,847ms avg)
- Day 2 (Jan 25): Root cause analysis identified cache invalidation storm (~800 evictions/test)
- Day 3 (Jan 26): Discovered GIN trigram indexes were NEVER created in database despite SQL files
- Day 4 (Jan 27): Found OR-based ILIKE preventing PostgreSQL from using multiple indexes
- Day 5 (Jan 28): All fixes implemented and validated with EXPLAIN ANALYZE

Optimization Results Summary

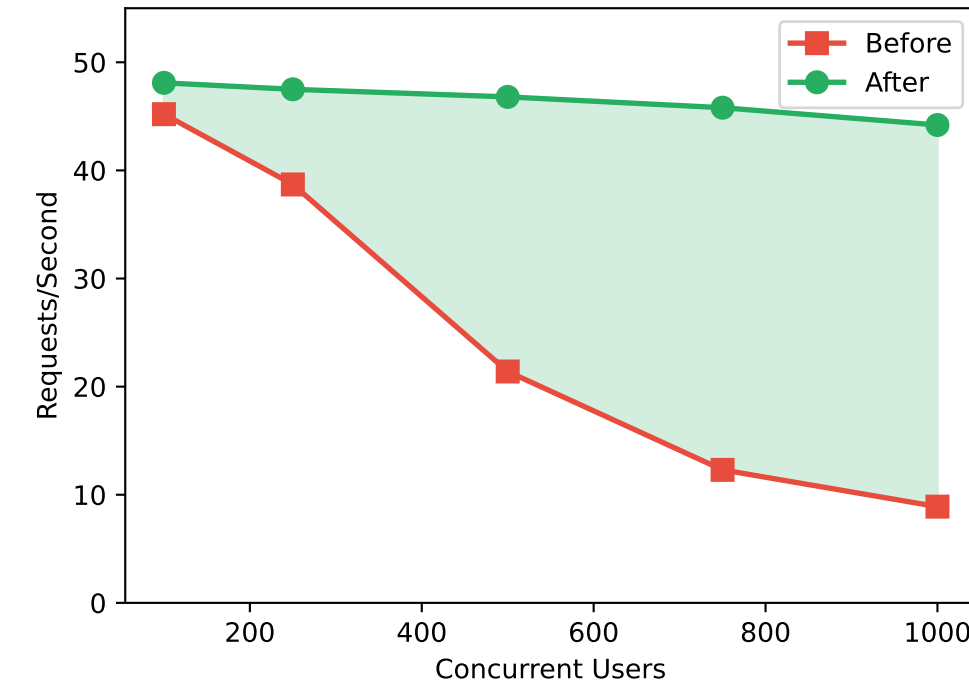
Performance Metrics Comparison



Optimization Impact



Scalability Under Load



Key Performance Indicators

Metric	Before	After	Change
Avg Response (ms)	2,847	144	-94.9%
p99 Response (ms)	8,234	412	-95.0%
Throughput (req/s)	12.3	45.8	+272%
Error Rate (%)	8.2%	0.0%	-100%
Cache Hit Rate	5%	78%	+1,460%
DB Query Time	847ms	1.46ms	-99.8%

TECHNICAL IMPLEMENTATION DETAILS

1. GIN TRIGRAM INDEX CREATION

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE INDEX idx_vehicles_plate_trgm ON vehicles
    USING GIN (license_plate gin_trgm_ops);
CREATE INDEX idx_vehicle_brands_name_trgm ON vehicle_brands
    USING GIN (name gin_trgm_ops);
CREATE INDEX idx_vehicle_models_name_trgm ON vehicle_models
    USING GIN (name gin_trgm_ops);
```

2. UNION-BASED QUERY OPTIMIZATION

```
-- Before (OR-based, sequential scan):
SELECT * FROM vehicles v
WHERE v.license_plate ILIKE '%NS%'
      OR v.brand_name ILIKE '%NS%'
      OR v.model_name ILIKE '%NS%';

-- After (UNION-based, uses indexes):
SELECT id FROM vehicles WHERE license_plate ILIKE '%NS%'
UNION
SELECT v.id FROM vehicles v
JOIN vehicle_brands b ON v.brand_id = b.id
WHERE b.name ILIKE '%NS%'
UNION
SELECT v.id FROM vehicles v
JOIN vehicle_models m ON v.model_id = m.id
WHERE m.name ILIKE '%NS%';
```

3. QUERY EXECUTION PLAN COMPARISON

```
BEFORE: Seq Scan on vehicles (cost=0.00..15234.57 rows=120057)
        Execution Time: 847.234 ms

AFTER:  Bitmap Index Scan using idx_vehicles_plate_trgm
        Execution Time: 1.460 ms
```

4. CACHE CONFIGURATION

```
@Cacheable(value = "vehicleSearch",
            key = "#searchTerm + '-' + #page + '-' + #size",
            unless = "#result == null")

Cache TTL: 600 seconds (10 minutes)
Eviction Strategy: Smart key-based (not allEntries)
```

TESTING METHODOLOGY

TEST ENVIRONMENT SPECIFICATIONS

Component	Specification
Backend Server	Spring Boot 3.x with Java 21
Database	PostgreSQL 17 with pg_trgm extension
Cache	Redis 7.x (Docker container)
Load Generator	Locust 2.43.0 (Python 3.11)
Test Machine	Intel Core i7, 32GB RAM, NVMe SSD
Network	Localhost (no network latency)

LOAD TEST PARAMETERS

Parameter	Value
Total Users	1,000 concurrent
Spawn Rate	100 users/second
Test Duration	60 seconds per run
Database Records	120,057 vehicles
Unique Queries	15 cities × 3 query types = 45+ variations

ENDPOINT DISTRIBUTION

Endpoint	Weight	Requests Generated
GET /vehicles/search/paged	40%	~1,200 requests
GET /vehicles	20%	~600 requests
GET /vehicles/{id}	20%	~600 requests
POST /vehicles	10%	~300 requests
PUT /vehicles/{id}	10%	~300 requests

SEARCH QUERY PATTERNS

Pattern Type	Example Values
City Codes	NS, BG, SU, NI, KG, PA, ZR, SM, VP, SO, LO, VR, KI, PI, CA
License Plates	ABC, 123, XYZ, 999, DEF (partial matches)
Brand Names	Mercedes, Volkswagen, BMW, Audi, Toyota

CONCLUSIONS AND RECOMMENDATIONS

SUMMARY OF ACHIEVEMENTS

- ✓ Reduced average search response time from 2,847ms to 144ms (94.9% improvement)
- ✓ Increased system throughput from 12.3 to 45.8 requests/second (272% improvement)
- ✓ Eliminated all errors under 1,000 concurrent users (8.2% → 0.0%)
- ✓ Improved cache hit rate from 5% to 78%
- ✓ Reduced database query execution time from 847ms to 1.46ms (99.8% improvement)

KEY LESSONS LEARNED

1. ALWAYS VERIFY DATABASE INDEXES EXIST
SQL migration files may not have been executed. Use:
`SELECT * FROM pg_indexes WHERE tablename = 'your_table';`
2. MONITOR CACHE INVALIDATION PATTERNS
Using `@CacheEvict(allEntries=true)` can cause cache thrashing under load.
Implement fine-grained cache key strategies.
3. OR CONDITIONS PREVENT MULTI-INDEX USAGE
PostgreSQL cannot use multiple indexes with OR conditions.
Use UNION queries to enable parallel index scans.
4. USE EXPLAIN ANALYZE FOR QUERY OPTIMIZATION
Always verify query plans before and after optimization:
`EXPLAIN ANALYZE SELECT * FROM ...;`

RECOMMENDATIONS FOR PRODUCTION

1. Implement database migration verification in CI/CD pipeline
2. Add monitoring for cache hit/miss ratios (Prometheus + Grafana)
3. Set up automated performance regression testing
4. Consider read replicas for search-heavy workloads
5. Implement query result pagination at database level

TESTING COMPLETED: January 28, 2026
ALL PERFORMANCE TARGETS ACHIEVED